

Relazione del Progetto di Programmazione di Reti - Traccia 1

Lorenzo Antonioli

2 maggio 2025

Indice

1	Introduzione	2
2	Architettura del Server	3
2.1	Struttura del Codice	3
3	Implementazione	4
3.1	Configurazione di Base	4
3.2	Logging delle Richieste	4
3.3	Generazione delle Risposte HTTP	4
3.4	Ciclo Principale del Server	5
4	Caratteristiche Implementate	7
4.1	Requisiti Minimi	7
4.2	Funzionalità Aggiuntive	7
5	Utilizzo del Server	8
5.1	Prerequisiti	8
5.2	Avvio del Server	8
5.3	Accesso al Server	8
5.4	Arresto del Server	8

Capitolo 1

Introduzione

Questo documento descrive l'implementazione di un web server HTTP scritto in Python utilizzando il modulo `socket`. Il server è in grado di: gestire richieste HTTP di base e i MIME types, fornire il logging delle richieste e servire file statici come HTML, CSS e immagini.

Capitolo 2

Architettura del Server

Il web server implementa un'architettura client-server di base seguendo il protocollo HTTP. È progettato per servire contenuti statici da una directory predefinita e gestisce richieste GET, rispondendo con codici di stato appropriati (200 OK, 404 Not Found, 405 Method Not Allowed).

2.1 Struttura del Codice

Il server è organizzato in tre componenti principali:

- Una funzione di logging per monitorare le richieste
- Una funzione per generare risposte HTTP
- Una funzione principale che gestisce il ciclo di vita del server

Capitolo 3

Implementazione

3.1 Configurazione di Base

Il server è configurato per funzionare su localhost (127.0.0.1) sulla porta 8080. Tutti i file statici sono serviti da una directory chiamata `www` nella stessa posizione del file del server.

```
1 HOST = 'localhost' # Il server ascolta solo su localhost
2 PORT = 8080        # Porta su cui ascolta il server
3 DIRECTORY = 'www'  # Cartella dove sono i file
```

Listing 3.1: Configurazione del server

3.2 Logging delle Richieste

Per facilitare il debug e il monitoraggio, il server registra ogni richiesta con un timestamp, il metodo HTTP utilizzato, il percorso richiesto e il codice di stato della risposta.

```
1 def log_request(method, path, status_code):
2     print(f"[{datetime.now()}] {method} {path} -> {
    status_code}")
```

Listing 3.2: Funzione di logging

3.3 Generazione delle Risposte HTTP

La funzione `generate_response` è responsabile della creazione delle risposte HTTP. Gestisce:

- La ricerca del file richiesto nel filesystem

- La determinazione del tipo MIME appropriato
- La generazione di intestazioni HTTP corrette
- La restituzione di risposte 404 per risorse non trovate

```

1 def generate_response(path):
2     full_path = os.path.join(DIRECTORY, path.lstrip('/'))
3
4     # Se e' una cartella, prova a servire index.html
5     if os.path.isdir(full_path):
6         full_path = os.path.join(full_path, 'index.html')
7
8     if os.path.exists(full_path) and not os.path.isdir(
9         full_path):
10        # File esistente -> restituisce 200 OK
11        with open(full_path, 'rb') as f:
12            content = f.read()
13            mime_type, _ = mimetypes.guess_type(full_path)
14            return b"HTTP/1.1 200 OK\r\n" + \
15                f"Content-Type: {mime_type or 'application/"}
16            octet-stream'}\r\n\r\n".encode() + \
17                content, 200
18        else:
19            # File non trovato -> 404 Not Found
20            content = b"<h1>404 Not Found</h1>"
21            return b"HTTP/1.1 404 Not Found\r\nContent-Type: text
22            /html\r\n\r\n" + content, 404

```

Listing 3.3: Funzione per generare risposte HTTP

3.4 Ciclo Principale del Server

La funzione `start_server` inizializza il socket TCP, ascolta le connessioni in entrata e gestisce le richieste dei client. Implementa anche una gestione sicura dell'interruzione tramite `KeyboardInterrupt` (CTRL + C).

```

1 def start_server():
2     # Crea un socket TCP
3     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM
4     )
5     server.bind((HOST, PORT)) # Assegna IP e porta
6     server.listen(1)
7     print(f"Web Server is up on http://{HOST}:{PORT}")
8     try:
9         while True:
10             client, address = server.accept()

```

```

10         with client:
11             request = client.recv(1024).decode()
12             if not request:
13                 continue
14             request_line = request.splitlines()[0] #
15             Prende solo la prima riga
16             method, path, _ = request_line.split()[:3] #
17             Estrae metodo (GET), percorso, protocollo
18
19             if method == 'GET':
20                 response, status = generate_respons(path)
21                 log_request(method, path, status)
22                 client.sendall(response) # Invia la
23                 risposta al browser
24             else:
25                 # Metodo non supportato (es: POST)
26                 response = b"HTTP/1.1 405 Method Not
27                 Allowed\r\n\r\n"
28                 client.sendall(response)
29                 log_request(method, path, 405)
30     except KeyboardInterrupt:
31         print('\n[!] CTRL+C revealed. Server closing...')
32     finally:
33         server.close()
34         print('[*] Server correctly closed')

```

Listing 3.4: Funzione principale del server

Capitolo 4

Caratteristiche Implementate

4.1 Requisiti Minimi

Il server implementa tutti i requisiti minimi specificati:

- Risponde su localhost:8080
- È in grado di servire pagine HTML statiche
- Gestisce richieste GET con codice di stato 200
- Implementa risposte 404 per file inesistenti

4.2 Funzionalità Aggiuntive

Oltre ai requisiti minimi, il server implementa anche:

- Gestione dei MIME types per diversi tipi di file
- Logging delle richieste
- Chiusura pulita del server in caso di interruzione

Capitolo 5

Utilizzo del Server

5.1 Prerequisiti

Per eseguire il server è necessario:

- Python 3.x installato
- Una directory `www` nella stessa posizione del file del server
- File HTML/CSS/immagini da servire nella directory `www`

5.2 Avvio del Server

Per avviare il server, eseguire il file Python da terminale:

```
1 python server.py
```

5.3 Accesso al Server

Una volta avviato, il server è accessibile tramite browser all'indirizzo:

```
1 http://localhost:8080
```

5.4 Arresto del Server

Per arrestare il server in modo sicuro, premere CTRL+C nel terminale. Il server gestirà correttamente l'interruzione e chiuderà tutte le risorse.